Nicole Snyder
04-17-2026
Professor Hicham Elhayboubi

<Draw It or Lose It>
CS 230 Project Software Design Template
Version 1.0

Table of Contents

CS 230 Project Software Design Template 1
Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 3
Evaluation 4
Recommendations 5

Document Revision History

Version	Date	Author	Comments
1.0	<03/20/2026>	<Nicole-Snyder>	<Completion of the Executive Summary, Design Constraints and Domain Model>
2.0	<04/03/2/2026>	<Nicole-Snyder>	<Completion of the Evaluation>

3.0	<04-17-2026>	<Nicole-Snyder>	<Completion of Recommendations>

Instructions
Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

The Gaming Room is moving its popular Android game, "Draw It or Lose It," to a web-based platform in order to support users across a range of devices. The main challenge here is providing smooth multi-platform access while maintaining a single, consistent game state in memory. This is crucial for preventing duplicate games, teams, or players, and for avoiding naming conflicts. To address these needs, the solution uses a robust Object-Oriented design that incorporates the Singleton and Iterator design patterns. The Singleton pattern ensures that there's only one, globally accessible game service managing all active games and unique identifiers. Meanwhile, the Iterator pattern makes it possible to safely go through game entities and enforce unique naming. A new Entity base class also brings efficiencies by encouraging inheritance for shared attributes, making the backend both efficient and scalable.

Requirements

< Please note: While this section is not being assessed, it will support your outline of the design constraints below. *In your summary, identify each of the client's business and technical requirements in a clear and concise manner.*>

Design Constraints

Building the game in a web-based, distributed environment brings its own set of challenges, especially around memory management, handling multiple users at once (concurrency), and network reliability. Since users from different platforms may interact with the game at the same time, the server needs to maintain a single, authoritative version of the game state. The Singleton pattern serves this need by making sure only one instance of the game engine operates in memory, preventing conflicts. Concurrency is another key concern: when several users try to create teams or games at the same time, safely iterating over collections with the Iterator pattern is essential to avoid errors like ConcurrentModificationException and to keep the data accurate. Plus, because web-based apps rely on stateless HTTP and can face network latency, the application needs to keep its memory usage optimized and well-organized for quick data access and smooth synchronization for everyone playing.

System Architecture View

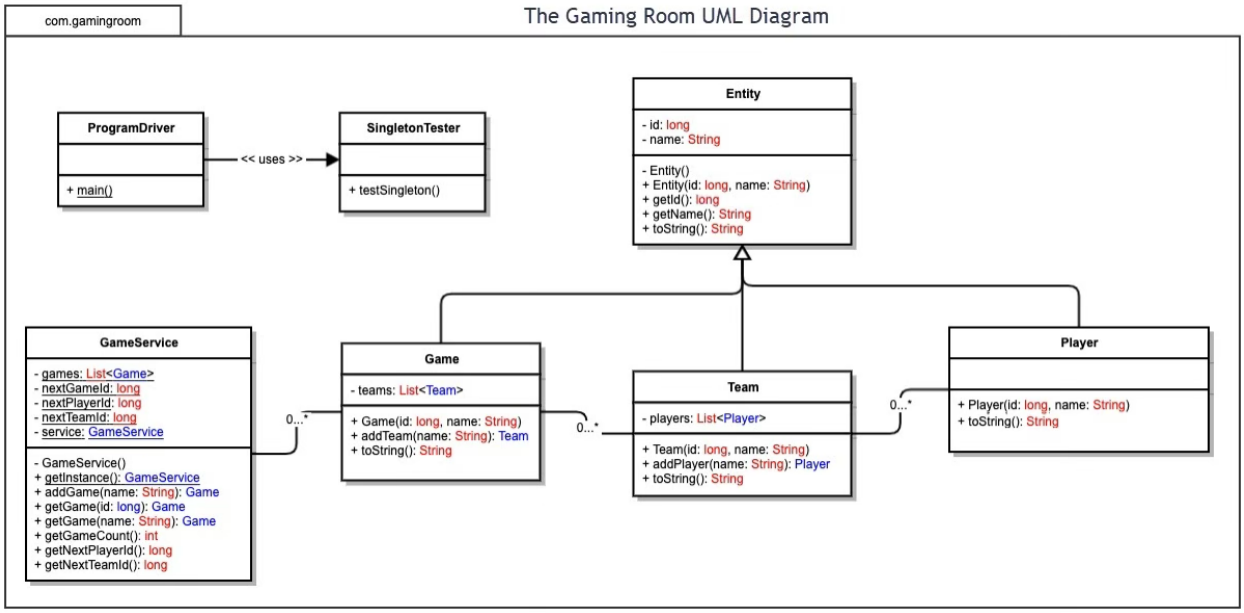
Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

The UML class diagram included with the application maps out both the hierarchy and relationships in "Draw It or Lose It." At the top, the GameService acts as the main manager, keeping track of a list of Game objects (shown in UML as a 0..* "contains" relationship). Each Game, in turn, manages its own list of Team objects, and every Team keeps a list of Player objects. This structure creates a clear and scalable organization for all game entities.

Key Object-Oriented Programming (OOP) principles demonstrated in the diagram include:

1. **Inheritance:** The Game, Team, and Player classes all inherit from a shared Entity base class (as indicated by the open-arrow lines in the UML diagram). This approach efficiently meets software requirements by letting these classes reuse common attributes like id and name, along with their getter methods. As a result, it significantly cuts down on code duplication and promotes cleaner, more maintainable code.
2. **Encapsulation:** Attributes like games, teams, players, id, and name are kept private within their classes. Access to these values, and the ability to change them, is controlled through dedicated public methods such as addTeam, addPlayer, and getName(). This protects the internal state of the classes from unwanted external changes, ensuring the integrity of the application.
3. **Software Design Patterns:** The GameService class is a textbook example of the Singleton pattern in action, as shown by its private constructor, static instance, and the public getInstance() method. This setup ensures that only one instance of GameService can ever exist, making it easy to manage the game state centrally for all users.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	macOS is typically not chosen for enterprise-level server hosting. Although it is technically possible to run web servers on macOS, Apple has phased out its macOS Server	Linux stands out as the industry standard for hosting web-based applications and is highly recommended for this project. Its strengths lie in its outstanding	Windows Server is a strong and reliable choice for hosting web applications, providing powerful deployment tools such as Internet Information Services (IIS) and smooth integration	Mobile devices, including iOS and Android platforms, are not suitable for server-side hosting. These devices are fundamentally designed to function as clients rather than servers. They lack the

	application, resulting in fewer built-in deployment and management tools compared to other platforms. Another significant limitation is hardware scalability. Expanding server resources requires investment in proprietary Apple hardware, which is generally more costly and less flexible than cloud-based alternatives. While there are no separate operating system licensing fees (since macOS comes with the hardware), the high upfront hardware costs and limited scalability make macOS a poor fit for hosting applications that anticipate thousands of concurrent users.	scalability, robust security, and high performance. Linux offers remarkable flexibility in deployment, supporting technologies like Docker containers, virtual machines, and cloud environments such as AWS, Azure, and Google Cloud. This makes it an excellent choice for applications that need to scale to thousands of users. Cost is another major benefit: most Linux distributions (such as Ubuntu or Debian) are open-source and free from licensing fees. Even enterprise-focused distributions, like Red Hat, charge for support rather than for per-user licenses, making Linux a very cost-effective solution for large-scale deployments.	with Microsoft Azure, which makes scaling up to thousands of users straightforward. The platform is robust, secure, and designed to be user-friendly for system administrators. However, the primary drawback is cost. Unlike Linux, Windows Server requires the purchase of software licenses, which can be costly and are typically structured around the number of processor cores or client access licenses (CALs). This can make Windows Server a more expensive option, especially for large-scale deployments.	necessary processing power, memory, persistent network connectivity, and deployment frameworks to support hosting web applications, especially at the scale required for thousands of concurrent users. As a result, there are no viable server deployment strategies or licensing considerations for these platforms in a hosting context.
Client Side	To ensure a smooth experience for Mac users, the application needs to work seamlessly with macOS desktop browsers, especially Apple Safari, as well as Google Chrome and Firefox. Developing for these platforms relies on standard web technologies like HTML5, CSS3, and JavaScript, so the required expertise is quite common. While the overall development time	Supporting Linux users is generally straightforward and cost-effective. The main requirement is to test the interface on browsers commonly used in Linux environments, especially Mozilla Firefox and Google Chrome. As with other platforms, standard web development skills are all that's needed. Since Linux can be run in virtual machines at no cost, compatibility testing	Given that Windows has the largest desktop user base, ensuring compatibility here is essential. The application should function perfectly on Microsoft Edge, Google Chrome, and Firefox. Since users will access the application through a responsive web interface rather than a downloaded program, development time and costs remain	Supporting mobile devices requires strict adherence to **Mobile-First responsive design** best practices to ensure the interface adjusts automatically to various screen sizes and orientations. This requires specific web expertise in modern layout techniques like **CSS Flexbox and Grid** to create fluid interfaces, moving beyond a basic command of **CSS Media Queries**. Additionally, developers must be

	and cost are modest, it's important for the team to have access to Mac hardware for thorough Safari testing, since Safari can behave differently from Chromium-based browsers.	is simple and inexpensive, keeping development time and costs low.	manageable. Standard web development expertise is sufficient, and testing can be performed easily on most development machines.	proficient in **touch-target optimization,** ensuring interactive elements are sized and spaced for fingers rather than a mouse, and **performance profiling** to ensure the game state remains synchronized despite the processing, memory, and network latency limitations typical of mobile hardware.
Development Tools	The project utilizes a standard web stack: **Java** for the backend and **HTML/CSS/JS** for the frontend. By using cross-platform IDEs like **Eclipse** or **VS Code**, Mac developers remain perfectly synced with the rest of the team, allowing for a **unified team structure** where code can be reviewed across different operating systems without compatibility issues	Utilizing free, open-source IDEs like **NetBeans** or **Eclipse** ensures that licensing costs remain at zero for the entire organization. Because these tools are identical to those used on Windows and Mac, the **full-stack team** can easily transition between environments, promoting a flexible workflow where any developer can troubleshoot server-side issues.	By opting for **Eclipse** or **Visual Studio Code** over expensive enterprise licenses, the project avoids per-developer software costs. This choice reinforces a **cross-functional team structure**; since everyone uses the same toolset, there is no need for a "Windows-only" sub-team, which significantly reduces communication overhead and project silos.	The use of a "**responsive HTML interface**" allows the existing **full-stack web team** to manage mobile views using **Chrome Developer Tools** for simulation. By avoiding native languages like Kotlin or Swift, the project eliminates the need to hire and manage **two additional specialized mobile teams**, ensuring a single code update applies to all devices.

<u>Recommendations</u>

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**: For the expansion of "Draw It or Lose It" to a multi-platform, web-based environment, **Linux** is the recommended server-side operating platform. Linux is the industry standard for hosting web applications due to its superior **scalability**, **security**, and **high performance**. It offers the flexibility to deploy across various environments, including **Docker containers**, **virtual machines**, and major **cloud providers** like AWS, Azure, and Google Cloud. This ensures the game can reliably scale to support thousands of concurrent users across Windows, Mac, and mobile clients. Additionally, most Linux distributions are **open-source**, significantly reducing licensing costs compared to Windows Server

2. **Operating Systems Architectures**: The recommended Linux platform utilizes a **monolithic kernel architecture**, where all OS services (like memory management and file systems) run in kernel space for maximum performance. In the context of "Draw It or Lose It," the system will follow a **client-server architecture**. The central Linux server acts as the authoritative host for the game engine and state, while diverse devices—Mac desktops, Windows PCs, and mobile phones—function as **clients** accessing the application through a **web browser**. This architecture supports multi-user and multi-tasking environments, allowing the server to handle many player requests simultaneously while maintaining a single game instance.

3. **Storage Management**: To manage the structured data of "Draw It or Lose It," a **Relational Database Management System (RDBMS)** such as **MySQL** or **PostgreSQL** should be implemented on the Linux server. Linux's **Virtual File System (VFS)** allows the OS to interact with these storage systems uniformly, ensuring efficient data retrieval. This storage solution is ideal for mapping the hierarchical relationships defined in the **Domain Model**, where the system must store and relate **Game**, **Team**, and **Player** entities. Furthermore, using a database ensures that **unique identifiers** (ids) are persisted correctly, supporting the requirements of the **Singleton** game service.

4. **Memory Management**: The Linux platform uses **Virtual Memory management**, employing techniques like **paging** and **swapping** to optimize RAM usage across concurrent processes. At the application level, "Draw It or Lose It" utilizes the **Singleton design pattern** to ensure only one instance of the GameService exists in the server's memory. This is critical for maintaining a single, consistent game state and preventing memory conflicts between users. To handle multiple users without errors, the **Iterator pattern** is used to safely navigate memory-resident collections of games and teams, preventing ConcurrentModificationException during high-traffic periods.

5. **Distributed Systems and Networks**: Communication between various client platforms and the server is achieved through **distributed software** using the **stateless HTTP protocol**. Because users are geographically dispersed, the application relies on a network that connects heterogeneous devices (Windows, Mac, Mobile) to the central Linux host. To handle **network latency** and potential **outages**, the system should implement **redundancy** through cloud-based availability zones and **load balancing**. This distributed approach ensures that even if one network node fails, the central game state remains synchronized and accessible to the rest of the player base.

6. **Security**: Security is addressed through both the operating platform and the software design. **Linux** provides robust security features, including **user permissions** (root/sudo) and **built-in firewalls** to protect the server from unauthorized access. To protect user information in transit, **TLS/HTTPS encryption** must be used for all communication between client browsers and the server. Internally, the application uses **Encapsulation;** keeping attributes like player names and IDs **private** to protect the integrity of the game data from unwanted external changes. This multi-layered approach ensures that sensitive player information remains secure across all platforms.